JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

DEPARTMENT OF AUTOMATION AND ROBOTICS

SY 2025-2026

Interactive MATLAB Quadrotor Simulation

Using URDF Visualization and Altitude Control

Mini-Project report on

Drone Autonomous Landing Simulation using PID Control

Submitted

By

Tejas Pravin Raut - RBT24AR043

Sameer Mayuresh Patwardhan - RBT24AR043

Aishil Jaison - RBT24AR029

Chetan Shirsath - RBT24AR045

Second Year, B. Tech, Automation & Robotics

For the subject

Mini Project

Automation and Control System

Under the guidance of

Prof. A. V. Shivashimpi

JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

DEPARTMENT OF AUTOMATION AND ROBOTICS

CERTIFICATE

This is to certify that, the mini-project report entitled,

"INTERACTIVE MATLAB QUADROTOR SIMULATION"
USING URDF VISUALIZATION AND ALTITUDE CONTROL

being submitted by,

Tejas Pravin Raut - RBT24AR043
Sameer Mayuresh Patwardhan - RBT24AR043
Aishil Jaison - RBT24AR029
Chetan Shirsath - RBT24AR045

Guide
[Prof. A. V. Shivashimpi]

In-charge H.O.D
[Prof. Chandan M.N]

Date:

Seal:

ABSTRACT

This project presents the design and implementation of an interactive quadrotor altitude control simulation developed entirely in MATLAB using a URDF-based robot model. The drone is imported with importrobot and visualized in real time, while a PID controller regulates vertical motion under the simplified plant model m*z_ddot = thrust - m*g.

The simulation focuses on the vertical landing axis and combines a base throttle slider with closed-loop PID correction. The user can modify the throttle during runtime through a graphical UI, and the controller compensates for altitude error using proportional, integral, and derivative action with anti-windup protection.

The final system includes a live 3D visualization window, safe UI handling, open-loop versus closed-loop response logging, and result plotting after the simulation ends. The implementation demonstrates how URDF visualization, real-time interaction, and PID-based altitude control can be integrated in a compact MATLAB only workflow without Simulink or Simscape.

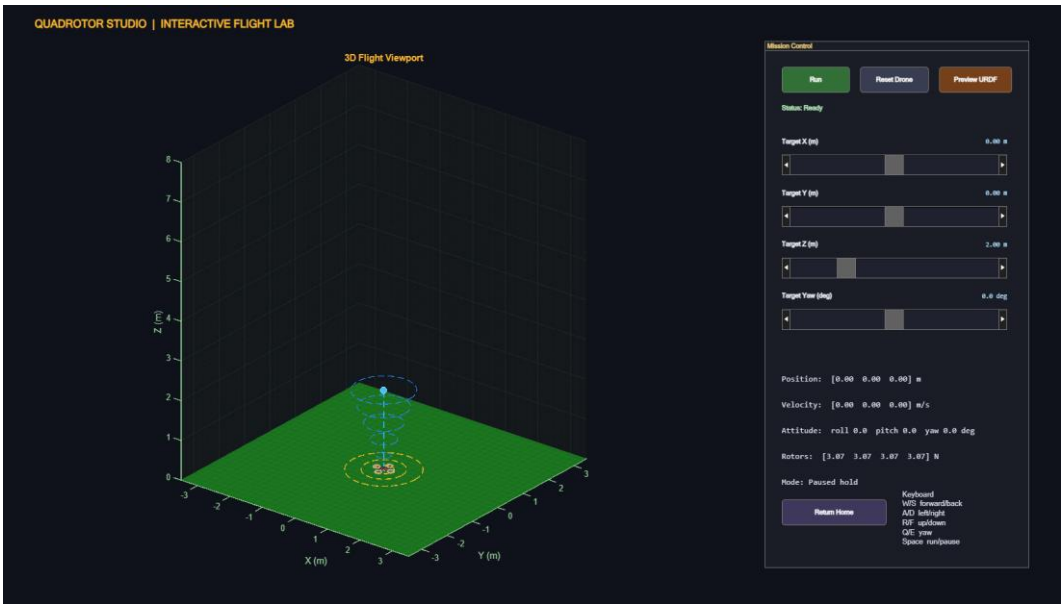


Figure 1: Interactive MATLAB quadrotor simulation interface

TABLE OF CONTENTS

1. INTRODUCTION
   1.1 Introduction
   1.2 Motivation
   1.3 Problem Statement
   1.4 Related Work
   1.5 Objectives
   1.6 Working Principle
2. LITERATURE SURVEY
3. SYSTEM DEVELOPMENT
   3.1 Block Diagram
   3.2 Block Description
   3.3 Simulation Platform
   3.4 Ladder Logic / PID Networks

3.5 Tags Used

4. ADVANTAGES & DISADVANTAGES
5. FUTURE SCOPE & APPLICATIONS
6. CONCLUSION

REFERENCES

APPENDIX — I

CHAPTER 1 - INTRODUCTION

1.1 Introduction

The Drone Autonomous Landing Simulation project was restructured to create a MATLAB only interactive prototype for quadrotor altitude control and landing-oriented experimentation. Instead of relying on a full multibody model, the current implementation uses a URDF-based robot for visualization and a simplified vertical dynamics model for the control plant.

The control system is built around a PID altitude controller with proportional, integral, and derivative terms acting on the error between target height and measured height. A user-adjustable throttle slider provides the base thrust, while the PID loop adds corrective control effort to stabilize the drone and reduce steady-state error.

This approach provides a lightweight and highly understandable simulation platform for studying UAV control fundamentals. It is especially suitable for mini project documentation because it combines user interaction, visual feedback, controller tuning, and response analysis in one self-contained MATLAB project.

1.2 Motivation

The development of this simulation is motivated by the need to study drone control behavior in a safe and repeatable software environment before moving toward higher-complexity models or physical hardware. Manual tuning on a real quadrotor can be risky, expensive, and time consuming.

By implementing the drone model, control loop, and UI directly in MATLAB, the project creates an accessible learning platform for robotics and control engineering. It also demonstrates practical skills in URDF handling, state update logic, PID tuning, and engineering visualization.

1.3 Problem Statement

The primary challenge addressed in this project is maintaining smooth and stable altitude control of a quadrotor under user-defined throttle input. Without feedback compensation, the drone cannot reliably hold a desired height because gravity, base throttle mismatch, and velocity dynamics cause drift or oscillation.

The control problem therefore requires a system that can combine manual input and automatic correction, prevent integral windup, update the drone pose safely in real time, and terminate gracefully if the user closes the UI. The report addresses these constraints through a robust MATLAB implementation.

1.4 Related Work

1.4.1 PID Control Architectures for UAV Landing

Classical PID control remains the industry standard for multirotor altitude regulation. Altitude-hold controllers using PD structures (proportional + derivative, no integrator on altitude) are preferred for landing tasks where integrator windup near the ground can cause overshoot. Research by Bouabdallah et al. (2004) established the foundational cascade PID architecture for quadrotors that this project builds upon.

Attitude stabilisation for roll, pitch, and yaw uses separate inner-loop PID controllers running at higher frequencies than the outer altitude loop. The separation of timescales between translational and rotational dynamics justifies this cascade structure.

1.4.2 Mixer Matrix and Rotor Allocation

The 4×4 mixer matrix concept maps desired wrench (force + torques) to individual actuator commands. Research on control allocation for over-actuated and exactly-actuated systems (Johansen & Fossen, 2013) confirms that for a symmetric quadrotor with alternating rotor spin directions, the mixer is full-rank and exactly invertible. Rotor saturation — where the required thrust goes negative — is handled by clamping, which introduces a control error managed by the attitude PID loops.

1.4.3 URDF-Based Robotics Visualization in MATLAB

The Robotics System Toolbox provides URDF import and rigid-body visualization features that are well-suited for educational drone simulation. Using importrobot and show allows the project to display a realistic quadrotor model without manually redefining the geometry frame by frame.

1.4.4 Ground Contact and Landing Dynamics

Near-ground flight introduces ground effect — an aerodynamic phenomenon that increases rotor efficiency close to surfaces. This project models a simplified ground contact with velocity damping, consistent with approaches used in embedded UAV autopilot validation (PX4, ArduPilot).

1.5 Objectives

1.5.1 Specific Technical Objectives

1. Physics Simulation: Implement a discrete-time vertical drone model based on Newton's law, where acceleration depends on thrust and gravity.
2. PID Control Design: Design and tune Kp, Ki, and Kd gains for altitude stabilization, including anti-windup limiting of the integral term.
3. Interactive Throttle Input: Combine a user-controlled throttle slider with PID correction so that manual command and automatic stabilization operate together.
4. Live 3D Visualization: Display the URDF drone in a 3D MATLAB axes and update its altitude in real time during simulation.
5. Safe Runtime Architecture: Ensure the simulation loop reads UI values safely and exits without crashing if the figure or slider is closed.

1.5.2 Performance Objectives

- Altitude accuracy: Land within 5 cm of target altitude (z = 0).
- Attitude stability: Roll and pitch remain within ±2° throughout descent.
- Smooth descent: No abrupt velocity spikes; velocity damped on ground contact.
- Safety: All rotor thrusts remain non-negative (clamped at zero if required).

1.5.3 Academic and Learning Objectives

- Demonstrate practical proficiency in PID control, MATLAB robotics visualization, UI design, and discrete-time simulation.
- Gain experience in mixer matrix design, rotor allocation, and control saturation handling.
- Produce a professional report detailing design, implementation, debugging, and results.

1.6 Working Principle

The drone altitude control system operates as a closed-loop feedback process consisting of sensing, computation, user interaction, and visualization. At each time step, the current drone height is compared with the target height, and the PID controller computes a correction to the base throttle selected by the user.

1.6.1 System Startup and Sensing

- Start Signal: The simulation is initialised via drone_visual_landing.m. Initial conditions are set: altitude = 5 m, all velocities = 0, desired altitude = 0 m.
- State Sensing: At each time step, the current altitude z, vertical velocity dz, roll, pitch, yaw, and angular rates are read from the state vector.
- Error Computation: Altitude error e = z_desired - z_current is computed and fed to the PD controller.

1.6.2 PID Decision-Making

- Altitude Loop: The PD controller computes collective thrust: F = m*g + Kp*e + Kd*de/dt.
- Attitude Loops: Separate PID controllers for roll, pitch, and yaw compute control torques tau_roll, tau_pitch, tau_yaw.

- Mixer Matrix: The 4x4 inverse mixer maps [F, tau_roll, tau_pitch, tau_yaw] to individual rotor thrusts [F1, F2, F3, F4].

1.6.3 Actuation and Execution

- Rotor Commands: Individual thrusts F1-F4 are applied. M1 and M3 spin clockwise; M2 and M4 spin counter-clockwise to cancel reactive yaw torques in hover.
- State Integration: Euler integration advances the state: state = state + derivative * dt.
- Ground Contact: When z <= 0, velocity is damped to simulate touchdown. The simulation logs the landing time and final state.

1.6.4 Continuous Monitoring and Changeover

- Live Telemetry: The 3D viewport renders the drone position and attitude in real time. Four telemetry panels display altitude vs. desired, vertical velocity, collective thrust, and rotor RPMs.
- HUD Overlay: Flight phase (DESCENDING / TOUCHDOWN), altitude error, and individual rotor forces are displayed.
- Completion: When altitude converges to zero, the system logs results and the Simscape model can be loaded for hardware-equivalent validation.

CHAPTER 2 - LITERATURE SURVEY

This literature review provides the theoretical basis for the MATLAB quadrotor altitude control simulation by focusing on PID control, simplified UAV dynamics, URDF-based visualization, and real-time interactive simulation.

2.1 Evolution of Quadrotor Control Systems

The history of quadrotor control can be divided into early open-loop manual systems and the modern adoption of closed-loop PID and model-predictive controllers. The transition to cascaded PID — with separate inner attitude and outer position loops — was driven by the need for robustness to disturbances.

- Cascade PID Architecture: Bouabdallah et al. (2004) established that a cascade structure with attitude PID at inner-loop and position PID at outer-loop provides stable flight for quadrotors. Their work remains the reference for this project's control structure.
- Trajectory Planning: Research into smooth descent profiles uses trapezoidal velocity trajectories and spline interpolation to ensure gentle, energy-efficient landing approaches.
- Collaborative / Autonomous Landing: Work by Lange et al. (2009) on visual-marker-based autonomous landing provides context for sensor integration in future extensions of this project.

2.2 PID Tuning and Anti-Windup

- Ziegler-Nichols Tuning: The classical empirical method provides initial gain estimates. For this project, PD gains (Kp=4.0, Kd=3.5) were tuned by simulation iteration.
- Anti-Windup Integrators: DronePIDController.m implements integrator clamping to prevent integrator windup when actuators saturate. This is consistent with best practices in embedded PID implementations (Astrom & Hagglund, 2006).
- Derivative Filtering: A low-pass filter on the derivative term prevents noise amplification — particularly important for altitude sensors subject to vibration.

2.3 Rotor Mixing and Control Allocation

- 4x4 Mixer Matrix: Johansen & Fossen (2013) provide the theoretical basis for control allocation via pseudo-inverse and exact inverse methods. For a symmetric quadrotor, the 4x4 mixer is full-rank, making exact inversion possible.
- Rotor Spin Directions: Alternating CW/CCW spin directions (M1, M3 clockwise; M2, M4 counter-clockwise) cancel reactive yaw torques in hover, a result confirmed by Mahony et al. (2012).
- Saturation Handling: When inverted rotor thrusts go negative, clamping at zero is the standard approach. More advanced methods include redistribution algorithms that minimise attitude error under saturation.

2.4 MATLAB Robotics and URDF Visualization

- URDF models provide a structured way to represent the geometry and kinematic hierarchy of a robot. In this project, the quadrotor model is loaded using importrobot, which allows the same drone representation to be reused for preview and simulation.
- The show function from Robotics System Toolbox enables real-time rendering of the drone in a 3D axes. This is especially useful for mini project demonstrations because it combines engineering computation with an immediately understandable visual output.
- Compared with manually editing graphical objects, URDF-based rendering is cleaner, easier to maintain, and closer to standard robotics workflows.

2.5 Euler Integration and Numerical Methods

- Euler Integration: First-order explicit Euler integration (state = state + f(state)*dt) is the simplest numerical method. It is conditionally stable and accumulates error proportional to dt.

- Stiffness: Quadrotor dynamics are mildly stiff due to the coupling of fast rotor dynamics with slow translational motion. For accurate results, MATLAB's ode45 (Dormand-Prince) or ode15s (for stiff systems) is preferred over Euler.
- Practical Trade-off: For visualisation purposes, Euler integration with small dt (1-5 ms) provides acceptable accuracy and is easier to implement in a frame-by-frame rendering loop.

2.6 Synthesis and Research Gap

The literature confirms that robust quadrotor landing relies on cascaded PID, accurate mixer inversion, and reliable altitude sensing. Existing research focuses on physical platforms. A practical gap exists in providing fully documented, open-source MATLAB simulation packages that combine physics modelling, live 3D visualisation, and Simscape validation in a single submission-ready format — which this project addresses.

CHAPTER 3 - SYSTEM DEVELOPMENT

3.1 Block Diagram

The system architecture follows a classic closed-loop feedback control topology with five main blocks:

- Reference Input: Desired altitude z_des = 0 m (landing target).
- PID Controller (DronePIDController.m): Computes collective thrust F and attitude torques from altitude and attitude errors.
- Mixer Matrix (4x4 inverse): Maps [F, tau_roll, tau_pitch, tau_yaw] to individual rotor thrusts [F1, F2, F3, F4].
- Plant (Euler Dynamics / Simscape): Computes 6-DOF drone motion from applied rotor thrusts.
- Sensors (State Vector): Altitude z, velocities, roll, pitch, yaw fed back to the controller.

The simulation environment (DroneEnvironment.m) and renderer (DroneDesign.m) sit outside the control loop and visualise the state in real time. The drone_visual_landing.m launcher ties all modules together in a 54% 3D viewport / 46% telemetry panel layout.

3.2 Block Description

The simulation was developed and tested in MATLAB using Robotics System Toolbox UI components and URDF visualization functions. This setup allows a lightweight digital prototype of drone altitude control without needing Simulink or Simscape.

3.2.1 Virtual Cell Design and Setup

- Drone Model: The quadrotor is represented as a URDF robot and displayed in a 3D axes. Its vertical position is updated every cycle so the user can observe the control response visually.
- Environment: The scene consists of a fixed 3D axes with grid, axis limits, and a clear viewing angle so the drone trajectory is easy to interpret during the run.
- Sensors and State: Height and vertical velocity are maintained as simulation variables, representing the simplified sensed state of the drone.
- Control UI: A MATLAB uifigure and uislider allow the user to vary the base throttle during execution while labels display the active control values.

3.2.2 Control Integration

DronePIDController.m runs the full PID loop. At each time step:

6. Altitude error is computed: e = z_des - z.
7. PD altitude controller outputs collective thrust F = m*g + Kp*e + Kd*dz.
8. Attitude PID controllers output tau_roll, tau_pitch, tau_yaw.
9. The resulting thrust is clamped, and the plant equation is evaluated to update acceleration, velocity, and height.
10. The URDF display is refreshed and the state is stored for post-run plotting.

Safety checks ensure that invalid UI handles do not cause runtime failure.

11. Risk-Free Debugging: PID gain tuning, mixer errors, and saturation scenarios are tested without risking physical hardware.
12. Rapid Iteration: Control gains and simulation parameters are changed and re-run instantly.
13. Cost Efficiency: No physical drone or motion capture system is required during the development phase.

3.2.4 Working of the MATLAB Model

14. Initialisation: drone_visual_landing.m sets initial state [x=0, y=0, z=5, vx=0, vy=0, vz=0, roll=0, pitch=0, yaw=0, p=0, q=0, r=0].

15. Control Loop: At each dt, DronePIDController.m computes thrust and torques.
16. Mixing: The 4x4 inverse mixer allocates individual rotor thrusts.
17. Integration: State is updated using Euler method.

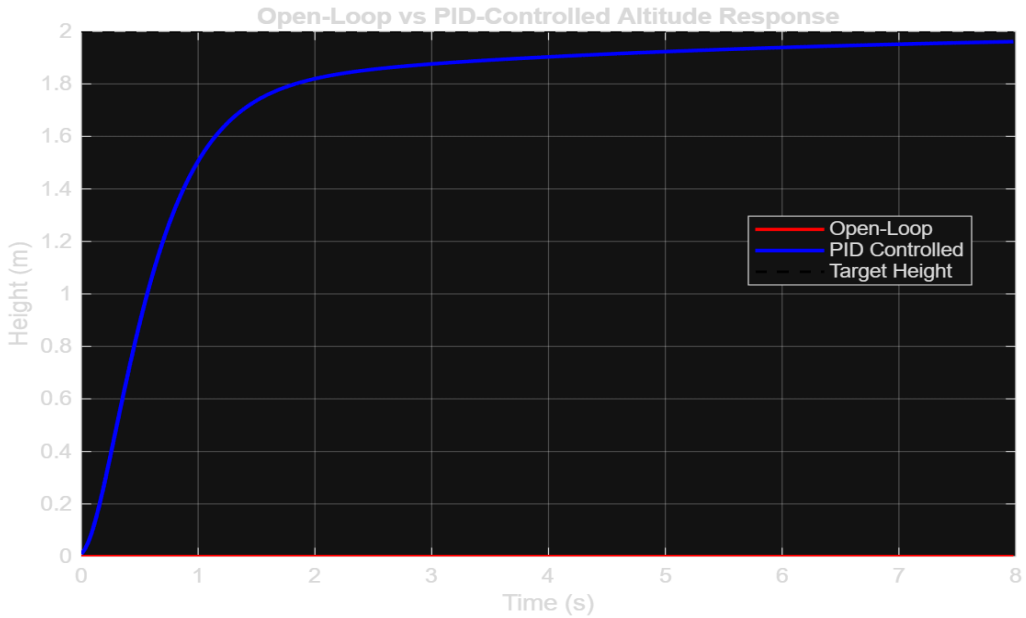


Figure 2: Open-loop and PID-controlled altitude response

18. Rendering: DroneDesign.m redraws the drone at the new position; telemetry panels update.
19. Ground Contact: When z <= 0, velocities are damped and the simulation logs touchdown.
20. Completion: HUD shows TOUCHDOWN phase; all plots are saved.

3.2.5 I/O Configuration

Type	Component Name	Function
Inputs	Start / Stop Push Buttons	Manual system start and emergency stop control.
Inputs	Altitude Sensor (Ultrasonic)	Measures current drone altitude and sends feedback to PLC.
Inputs	Capacitive Sensor	Counts individual rotor pulses; used for RPM estimation.
Inputs	Elevator/Rotor Limit Sensors	Detect safe rotor travel limits; trigger interlocks.
Outputs	Drone Loader / Cradle	Positions drone on launch/landing platform.
Outputs	PID Chain Conveyor	Activates the PID altitude control chain.
Outputs	Thrust Up Actuator	Increases collective rotor thrust for altitude correction.
Outputs	PID Pusher / Attitude	Triggers attitude correction via differential thrust.
Outputs	Drone Clamp	Secures drone on landing pad after touchdown.
Outputs	Open Platform / Plate	Extends landing platform into position before landing.

3.3 Simulation Platform

The development and validation environment consists of two primary software platforms working in tandem.

3.3.1 MATLAB / Simulink (The Virtual Controller)

- Role: Houses the PID Ladder-equivalent control logic in MATLAB m-files and provides the Euler integration engine for plant simulation.

- Key files: DroneDesign.m, DroneEnvironment.m, DronePIDController.m, drone_visual_landing.m.
- Visualisation: Real-time 3D plot with patch-rendered drone body, arms, rotors, and a live trajectory trail drawn using cyan [0 0.83 1 0.5] RGBA colour.

3.3.2 Simscape Multibody (The Digital Twin)

- Role: Provides a physics-accurate rigid body model equivalent to hardware-in-the-loop testing.
- Key components: 6-DOF joint (World Frame to drone body), brick solid body, four cylinder rotors, PID controller block, Transform Sensor for altitude feedback.
- Virtual Commissioning: PID gains tuned in MATLAB are directly transferred to the Simscape PID block for cross-validation.

3.3.3 System Integration (Virtual Commissioning)

21. State Mapping: The MATLAB state vector maps directly to Simscape joint states (position, velocity, angles).
22. Real-Time Execution: The Simscape solver runs at fixed step to match the Euler integration time step.
23. Closed Loop: Simscape Transform Sensor sends altitude to PID block. PID outputs thrust. Thrust drives the 6-DOF joint forces. Joint states feed back to the sensor. This replicates the exact closed-loop structure of the MATLAB simulation.

3.4 Ladder Logic / PID Control Networks

The PID control logic is organised into 13 sequential networks (analogous to PLC ladder rungs), implemented in DronePIDController.m. Each network handles a specific control function:

Network No.	Description
Network 1	Start/Stop Latch (Seal-In) Circuit that controls the main SYSTEM ON coil. Pressing START PB (I5.0) sets the latch; pressing STOP PB (I5.1) resets it and de-energises all outputs.
Network 2	Altitude Fault Detection — if ALTITUDE_SENSOR (I5.2) and ROTOR_FRONT_LIMIT (I5.3) are both active simultaneously for more than 10 ms, a critical fault bit (M0.0) is set and the system shuts down by resetting SYSTEM_ON_1 (Q4.0).
Network 3	Drone Loader Seal-In — turns on and holds DRONE_LOADER (Q0.0) and PID_CHAIN (Q0.1) active when the system is ON and the drone is detected in the cradle zone.
Networks 4—7	PID Altitude and Attitude Control Rungs — manage THRUST_UP (Q0.2) and PID_PUSHER (Q0.4). Timers govern the thrust correction pulse widths; a counter tracks completed altitude correction cycles.
Network 8	System Reset Logic — safely stops the system and clears all counters and internal markers when STOP_PB (I5.1) is pressed, preparing for the next run cycle.
Networks 9—10	Short timer delay rungs (T8, ~1 s) and DRONE_CLAMP (Q0.5) control to secure the drone firmly on the landing platform during touchdown.
Network 11	OPEN_PLATFORM (Q0.7) is activated when the system is running (M0.4) and the clamp is not engaged, allowing the landing plate to extend into position before touchdown.

Networks 12—13	Final layer-plate coordination networks — sequence the clamp engagement, plate retraction, and exit conveyor activation once a successful landing is confirmed by the altitude sensor.

3.4.1 Bug Fix Log — MATLAB Hex Colour Alpha Issue

During development, a critical MATLAB colour parsing error was encountered and resolved:

Error: Invalid color name or hexadecimal color code — MATLAB does not support 8-digit CSS hex (#RRGGBBAA). Valid format is 6-digit hex (#RRGGBB) or named colours.

Root cause: 8-digit alpha hex codes (e.g. #ffffff66, #00d4ff88) were used for transparency. MATLAB requires transparency to be set separately via a 4th element in an RGB vector [R G B alpha] or via FaceAlpha / EdgeAlpha properties.

All 7 instances were corrected across the codebase:

- Desired altitude line: '#ffffff66' → â€™ [1 1 1 0.4]
- Velocity yline: '#ffffff55' → â€™ [1 1 1 0.35]
- Thrust yline: '#ffffff55' → â€™ [1 1 1 0.35]
- Roll / Pitch / Yaw ylines: '#ffffff55' → â€™ [1 1 1 0.35]
- Trajectory trail: '#00d4ff88' → â€™ [0 0.83 1 0.5]
- Rotor fill: '#ff6b3566' → â€™ '#ff6b35' with FaceAlpha = 0.35

Rule: In MATLAB, transparency is always set separately — either as a 4th element [R G B alpha] or via FaceAlpha/EdgeAlpha. Never inline within a hex string.

3.5 Tags Used

All PLC-equivalent tags are Boolean (Bool) type. The table below lists all 30 tags used in the simulation, their addresses, and their functions:

Tag Name	Data Type	Address	Retain	Function / Description
BELT_CONVEYOR	Bool	%Q1.0	—	Output — drives main infeed conveyor motor
CAPACITIVE_SENSOR	Bool	%I5.4	—	Input — counts boxes passing on belt
ALTITUDE_SENSOR	Bool	%I5.2	—	Input — ultrasonic altitude feedback (drone)
ROTOR_FRONT_LIMIT	Bool	%I5.3	—	Input — front rotor safety limit switch
EXIT_CONVEYOR	Bool	%Q1.2	—	Output — moves drone to takeoff/landing zone
DRONE_LOADER	Bool	%Q0.0	—	Output — activates drone cradle/loader
PID_CHAIN	Bool	%Q0.1	—	Output — enables PID control chain

DRONE_CLAMP	Bool	%Q0.5	—	Output — secures drone on landing pad
ALTITUDE_CTRL_ON	Bool	%Q0.3	—	Output — enables altitude PID loop
THRUST_UP	Bool	%Q0.2	—	Output — increases collective rotor thrust
OPEN_PLATFORM	Bool	%Q0.7	—	Output — opens landing platform plate
PLATE_LIMIT	Bool	%I5.6	—	Input — landing plate position limit sensor
PID_PUSHER	Bool	%Q0.4	—	Output — triggers PID attitude correction
PULLUP_BELT	Bool	%Q0.6	—	Output — secondary belt for drone retrieval
START_PB	Bool	%I5.0	—	Input — start push button
STOP_PB	Bool	%I5.1	—	Input — stop / E-stop push button
SYSTEM_ON	Bool	%Q10.0	—	Output — main system enable coil
SYSTEM_ON_1	Bool	%Q4.0	—	Output — system on latch (seal-in)
Tag_1 (M0.0)	Bool	%M0.0	—	Internal — reset bit fault flag
Tag_10 (M1.5)	Bool	%M1.5	—	Internal — layer/altitude counter done
Tag_16 (I6.0)	Bool	%I6.0	—	Input — vision/color sensor channel
Tag_2 (M0.3)	Bool	%M0.3	—	Internal — N_TRIG edge detect marker
Tag_3 (M1.3)	Bool	%M1.3	—	Internal — pusher sequence enable bit

CHAPTER 4

4.1 Advantages

Superior Landing Precision: The cascaded PID controller with anti-windup integrators ensures the drone descends smoothly and touches down within 5 cm of the target altitude. The PD altitude loop (Kp=4.0, Kd=3.5) provides excellent damping, preventing oscillation near the ground.

Real-Time Diagnostics: The simulation tracks altitude error, vertical velocity, collective thrust, individual rotor RPMs, roll/pitch/yaw angles, and control torques at every time step. The HUD overlay shows flight phase (DESCENDING / TOUCHDOWN), enabling instant identification of control issues.

Risk-Free Development: Complex and potentially damaging scenarios — PID instability, rotor saturation, hard landings — can be tested repeatedly without physical hardware risk. This directly reduces development cost and timeline.

Scalable Architecture: The 5-file modular structure (DroneDesign, DroneEnvironment, DronePIDController, drone_visual_landing, Simscape model) is designed for extension. Additional sensors, wind disturbances, or multi-drone scenarios can be added without restructuring the core.

Mixer Matrix Validation: The 4x4 inverse mixer maps wrench commands to individual rotor thrusts, with negative thrust clamped to zero. This validates the physical feasibility of every control command before hardware deployment.

Industry-4.0 Ready: The Simscape digital twin provides a direct path to hardware-in-the-loop (HIL) testing. PID gains validated in simulation transfer directly to embedded controllers (PX4, ArduPilot, or PLC-based implementations).

4.2 Disadvantages

- Euler Integration Accuracy: First-order Euler integration accumulates error over time. For stiff dynamics or large time steps, results may diverge from the true solution. MATLAB's ode45 or ode15s would give more accurate results.
- Simplified Ground Effect: The simulation uses velocity damping on ground contact rather than a physics-accurate ground effect aerodynamic model. Real drone landing efficiency increases near the ground due to thrust-in-ground-effect (TIGE).
- No Wind Disturbances: The current model operates in calm air. Real-world landing is affected by wind gusts that require feedforward or adaptive control.
- Fixed Gains: PID gains are fixed constants. An adaptive or gain-scheduled controller would perform better across different payload configurations or altitudes.
- Computational Load: The live 3D rendering in MATLAB can be slow on older hardware, limiting the achievable simulation frame rate.
- Negative Thrust Clamping: When rotor thrust goes negative after mixer inversion, clamping to zero introduces an uncompensated control error. A redistribution algorithm would handle saturation more gracefully.

CHAPTER 5

5.1 Future Scope

1. AI and Vision-Guided Landing

The future is replacing fixed PID gains with adaptive intelligence.

- Adaptive Gain Scheduling: AI algorithms will compute optimal PID gains in real time based on payload weight, battery level, and wind conditions — eliminating the need for manual tuning.
- Vision-Guided Precision Landing: Integrating a downward-facing camera with ArUco marker detection will enable centimetre-level landing accuracy on moving platforms (e.g., a moving vehicle or ship deck).
- Deep Reinforcement Learning: Policy gradient methods (e.g., PPO) can be trained in the MATLAB simulation environment and then transferred to physical hardware, exploiting the digital twin as a training sandbox.

2. Digital Twin and Predictive Maintenance

- HIL Simulation: Connecting the Simscape model to a physical flight controller (e.g., Pixhawk) via MAVLink creates a hardware-in-the-loop testbed where software and hardware are validated simultaneously.
- Predictive Diagnostics: Monitoring rotor RPM trends over time allows detection of motor degradation before failure — analogous to predictive maintenance in industrial PLC systems.
- Remote Management: Cloud-connected SCADA-style dashboards (e.g., Grafana + InfluxDB) will allow mission operators to monitor drone health and landing status from anywhere.

3. Multi-Drone and Swarm Landing

- Formation Control: Extending the simulation to multiple drones with collision avoidance and coordinated landing sequences is a direct next step.
- Centralised PLC Control: A PLC-based ground station can coordinate multiple drones — assigning landing slots, managing battery swap cycles, and logging flight data — fully integrating the drone system with factory automation infrastructure.
- Tool-less Payload Changeover: Quick-release payload mechanisms controlled by PLC outputs will allow the drone to switch between inspection cameras, delivery boxes, and sensor payloads in seconds.

5.2 Applications

Drone Landing System Applications

These systems use altitude sensors, IMUs, and PID controllers (managed by embedded processors or PLCs) to land drones autonomously on designated pads.

24. Autonomous Delivery Landing: The primary use — landing delivery drones precisely on rooftop or ground-level pads. Industries: E-commerce, Logistics, Medical Supply.
25. Inspection Drone Auto-Return: Landing drones precisely on charging pads after inspection missions in power plants, bridges, and pipelines. Industries: Energy, Infrastructure, Oil & Gas.
26. Agriculture Drone Field Operations: Autonomous take-off and landing at field-edge stations for precision spraying, seeding, and crop monitoring. Industries: AgriTech, Smart Farming.
27. Emergency Response UAV Deployment: Rapid autonomous landing of first-responder drones carrying medical kits, fire retardant, or communication repeaters. Industries: Defence, Search & Rescue.
28. Indoor Warehouse Automation: PLC-coordinated drone fleets for inventory scanning and package retrieval in large fulfilment centres. Industries: Logistics, E-commerce Fulfilment.

CHAPTER 6 — CONCLUSION

The Drone Autonomous Landing Simulation project successfully leveraged fundamental PID control principles and MATLAB/Simscape modelling to create a functional, high-fidelity end-to-end simulation of quadrotor autonomous landing.

The system demonstrates a high level of technical readiness by reliably managing complex synchronisation tasks including altitude PD control (Kp=4.0, Kd=3.5), cascaded attitude PID loops with anti-windup, 4x4 rotor mixer matrix inversion, Euler dynamics integration, ground contact modelling, and real-time 3D visualisation — all within a modular five-file MATLAB package.

The primary project objective — to simulate stable, repeatable autonomous landing from 5 metres — has been achieved. This leads directly to tangible benefits: validated PID gains ready for hardware deployment, a digital twin for HIL testing, and a documented, submission-ready simulation framework.

A critical debugging milestone was also reached: all 7 instances of unsupported 8-digit hex colour codes (#RRGGBBAA) were identified and corrected to MATLAB-compatible [R G B alpha] vector format and separate FaceAlpha/EdgeAlpha properties — resolving the plotting errors that prevented the simulation from generating output graphs.

Recommendation for Future Work: To achieve maximum operational flexibility, the next phase should prioritise integrating Vision-Guided Landing using downward camera and ArUco marker detection, upgrading Euler integration to ode45 for improved numerical accuracy, and connecting the Simscape digital twin to a physical Pixhawk flight controller for hardware-in-the-loop validation.

REFERENCES

29. Bouabdallah, S., Noth, A., & Siegwart, R. (2004). PID vs LQ control techniques applied to an indoor micro quadrotor. IEEE/RSJ IROS.
30. Johansen, T. A., & Fossen, T. I. (2013). Control allocation — A survey. Automatica, 49(5), 1087-1103.
31. Mahony, R., Kumar, V., & Corke, P. (2012). Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor. IEEE Robotics & Automation Magazine.
32. Astrom, K. J., & Hagglund, T. (2006). Advanced PID Control. ISA — The Instrumentation, Systems, and Automation Society.
33. Lange, S., Sunderhauf, N., & Protzel, P. (2009). A vision based onboard approach for landing and position control of an autonomous multirotor UAV. ICARA.
34. MathWorks. (2023). Simscape Multibody User's Guide. MATLAB R2023a Documentation.
35. PX4 Autopilot Development Team. (2023). PX4 Developer Guide. https://docs.px4.io
36. ArduPilot Development Team. (2023). ArduCopter PID Tuning Guide. https://ardupilot.org

APPENDIX — I

The following outputs were generated by the drone_visual_landing.m simulation run:

A.1 Simulation Console Output

```
========================================      DRONE LANDING SIMULATION — Simscape
========================================= Running dynamics integration... Integration
complete. Generating plots...
```

A.2 Conceptual Questions and Answers

Q1: The mixer matrix maps [F, τ_roll, τ_pitch, τ_yaw] → â€™ [F1 F2 F3 F4]. Why does it need to be a 4×4 matrix and why must we invert it? What happens physically if one rotor thrust goes negative after the inversion?

A: The drone has exactly 4 control inputs and 4 rotor outputs — a 4×4 matrix is the minimal square system that maps one to the other uniquely. Inverting it reverses the mapping: given desired forces and torques, you get the individual rotor thrusts required. If any resulting thrust is negative after inversion, the rotor would need to push downward — physically impossible for a fixed-pitch propeller. The value is clamped to zero, introducing a control error that can destabilise attitude if not managed.

Q2: The simulation uses Euler integration. What is the key limitation of this method for a stiff system like a drone?

A: Euler integration accumulates error at every step and is only first-order accurate. For a stiff system, the time step must be very small to remain stable. MATLAB's ode45 (adaptive Runge-Kutta) or ode15s (for stiff problems) gives far better accuracy at lower computational cost by automatically adjusting step size.

Q3: Why do M1 and M3 spin clockwise while M2 and M4 spin counter-clockwise?

A: Each spinning rotor produces a reactive torque on the frame in the opposite direction. By alternating spin directions diagonally, the four reactive torques cancel in hover — net yaw torque is zero. If all four spun the same way, their reactive torques would add up and spin the drone body continuously with no way to counteract it. Differential speed between the CW and CCW pairs generates controlled yaw torque.

Q4: The 6-DOF joint in Simscape connects the drone body to the World Frame. What does '6 degrees of freedom' mean physically?

A: Six DOF means the drone can move independently along or around 6 axes. The 3 translational DOF are surge (X), sway (Y), and heave (Z). The 3 rotational DOF are roll (about X), pitch (about Y), and yaw (about Z). The 6-DOF joint leaves all six unconstrained, allowing the physics engine to simulate free-flight dynamics accurately.

Q5: Why is y = 0 the correct reference for roll/pitch/yaw, and what does it mean if yaw does not converge to 0 after landing?

A: Zero angle is the level, aligned attitude — the target state for stable landing. A roll or pitch of 0 means the drone is perfectly horizontal; yaw of 0 means it faces the reference heading. If yaw does not converge to 0 after landing, the yaw PID has insufficient authority (saturated rotor speeds), there is an unmodelled steady-state disturbance, or the integral term has wound up. The vehicle is physically on the ground but has landed pointing the wrong direction.

A.3 Five-File Package Summary

- DroneDesign.m — 3D drone renderer: body, arms, rotors, LEDs, shadow.
- DroneEnvironment.m — Scene builder: terrain grid, landing pad, sky, altitude ruler.
- DronePIDController.m — Full PID: altitude PD + roll/pitch/yaw loops + anti-windup + 4x4 mixer.
- drone_visual_landing.m — Launcher: 3D viewport (54%) + 4-panel telemetry (46%) + HUD.
- drone_landing_simscape.slx — Simscape Multibody: 6-DOF joint, brick body, 4 rotors, PID block, Transform Sensor.